A Catalogue of Silent Methodological Errors in scikit-learn ML Workflows: New Linting Rules for MLGuard

TL;DR

- The peer-reviewed literature documents a large, well-structured catalogue of **silent** ML errors — ones that run without crashing but inflate, invalidate, or render irreproducible the results — and the majority map cleanly onto static, scikit-learn-specific code signatures that MLGuard can detect; the single most authoritative organizing framework is the **Kapoor & Narayanan (2023, *Patterns*, DOI 10.1016/j.patter.2023.100804) eight-type leakage taxonomy** (L1.1–L3.3), which their survey of 22 review papers found affecting **at least 294 papers across 17 scientific fields**. (arxiv + 2)
- The highest-value NEW rules to add are: **imputation/PCA/feature-engineering fitted before split** (preprocessing leakage beyond scaling), **group/duplicate leakage** (random split of multi-record entities → use `GroupShuffleSplit`/`GroupKFold`), **SMOTE/resampling outside an `imblearn` Pipeline**, **target-encoding without cross-fitting**, **non-nested CV for hyperparameter tuning + test-set reuse**, **temporal leakage in time-ordered data**, and **multi-test leakage (no independent test set)** — each is statically detectable and each has direct peer-reviewed grounding.
- Prior static analyzers prove this is feasible: Yang et al. (ASE 2022, DOI 10.1145/3551349.3556918) detected overlap, multi-test, and preprocessing leakage at **92.9% accuracy / 97.6% precision / 67.8% recall** and found leakage in **29.6% of 107,603 public GitHub notebooks**, validating both the approach and the prevalence of these silent errors. (PubMed Central)

Key Findings

1. **Leakage is the dominant silent error and has a canonical taxonomy.** Kapoor & Narayanan's taxonomy is the backbone: **[L1.1]** no test set, **[L1.2]** preprocessing on train+test together, **[L1.3]** feature selection on train+test, **[L1.4]** duplicates across splits, **[L2]** illegitimate/proxy features, **[L3.1]** temporal leakage, **[L3.2]** non-independence between train/test, **[L3.3]** sampling bias in the test distribution. MLGuard's existing rules cover parts of L1.2/L1.3; L1.1, L1.4, L2, L3.1, and L3.2 are largely unaddressed and statically tractable. (arxiv) (arxiv)
2. **Preprocessing leakage generalizes far beyond `StandardScaler`.** The scikit-learn "Common pitfalls" page explicitly states the risk applies to "almost all transformations… including (but not limited to) `StandardScaler`, `SimpleImputer`, and `PCA`." Any transformer whose `fit`/`fit_transform` is called on data before the split (or outside a Pipeline within CV) is a leak. This is a *reduce-style* operation (computes across rows: mean, std, min/max, component vectors, category means, text vocabulary). (scikit-learn)

3. **Group/duplicate leakage is a distinct, common, and statically detectable error.** When multiple rows belong to one entity (patient, customer, subject), a random `train_test_split`/`KFold` puts the same entity in both train and test, inflating scores. The canonical example is the NIH **ChestX-ray14** dataset (Wang et al., CVPR 2017): exactly **112,120 frontal chest radiographs from 30,805 unique patients** (~3–4 images/patient), where random rather than patient-level splitting let models memorize patients. Fix: `GroupShuffleSplit`/`GroupKFold`/`StratifiedGroupKFold`. Duplicate/augmentation/oversampling before splitting (L1.4) is the same failure mechanism. GeeksforGeeks arxiv
4. **Class-imbalance handling has two silent failure modes.** (a) Resampling (SMOTE/over/under-sampling) applied before the split or before CV leaks synthetic neighbors across the split — the fix is `imblearn.pipeline.Pipeline`, which scikit-learn's own `Pipeline` cannot replace because resamplers expose `fit_resample`, not `transform`. (b) Using **accuracy** on imbalanced data (the "accuracy paradox") — though metric choice for imbalance is nuanced (recent work argues ROC-AUC is actually robust to imbalance while PR-AUC is sensitive). Towards Data Science + 2
5. **Hyperparameter tuning and test-set reuse cause optimistic bias** (Cawley & Talbot, JMLR 2010; Dwork et al. on the reusable holdout). Non-nested CV used for both selection and evaluation, and repeated/adaptive use of the test set, produce overoptimism that Cawley & Talbot show can be "of comparable magnitude to differences in performance between learning algorithms." Detectable patterns: `GridSearchCV.best_score_` reported as performance; test set used inside a tuning loop; `.score(X_test)` called repeatedly. JMLR
6. **Categorical encoding errors are silent and statically detectable.** `LabelEncoder`/`OrdinalEncoder` on nominal categories imposes a false order; target/mean encoding without cross-fitting causes catastrophic overfitting (scikit-learn's `TargetEncoder.fit_transform` uses internal cross-fitting precisely to avoid this — using `fit` then `transform`, or a naive groupby-mean merge, disables that protection).
7. **Temporal leakage and shortcut/Clever Hans learning** complete the picture. Random splits of time-ordered data (use `TimeSeriesSplit`; in finance, purged/embargoed CV per López de Prado) and confounder/proxy features (Clever Hans, e.g., hospital tokens in chest X-rays) are documented; the former is statically detectable, the latter mostly requires runtime/data inspection.
8. **Feasibility is proven.** Yang et al. (ASE 2022) detected overlap/multi-test/preprocessing leakage at high accuracy across 100k+ notebooks; Drobnjaković, Subotić & Urban built NBLyzer using abstract interpretation to prove the *absence* of leakage. MLGuard sits squarely in this validated tradition.

Details

Category A — Preprocessing / train-test contamination leakage (mostly STATICALLY

DETECTABLE)

A1. Imputation fitted on full data (before split or outside a CV pipeline).

- *Description:* `SimpleImputer`/`KNNImputer`/`IterativeImputer` computes fill values (mean, median, mode, neighbor-based) using the whole dataset, so test-set statistics flow into training.
- *Why silent:* Code runs; imputed values are valid numbers; only the inflated score betrays it.
- *Bad pattern:* `X_imp = SimpleImputer().fit_transform(X)` then `train_test_split(X_imp, y)`; or `imputer.fit_transform(X)` then `cross_val_score`.
- *Correct:* Put the imputer inside a `Pipeline`/`ColumnTransformer` passed to CV, or fit on `X_train` only. KNN-impute leakage is especially insidious because the neighbor computation crosses the split (Apicella et al., "Don't Push the Button," arXiv 2401.13796, §4.4.3, identify ⩾3 affected studies).
- *Detectability:* STATIC (fit/fit_transform on data that flows to both splits).
- *Sources:* scikit-learn Common Pitfalls; Apicella et al. 2024; Kapoor & Narayanan **L1.2**.

A2. Dimensionality reduction / feature extraction / feature selection fitted on full data.

- `PCA`, `TruncatedSVD`, `NMF`, `SelectKBest`, `SelectPercentile`, `RFE`, and text vectorizers (`CountVectorizer`, `TfidfVectorizer`) fitted before the split. scikit-learn names PCA explicitly; Yang et al. found preprocessing leakage in **13.4%** of notebooks doing PCA and **32.9%** of those that scale.
- *Correct:* fit inside Pipeline/CV on training data only.
- *Detectability:* STATIC.
- *Sources:* scikit-learn docs; Lones (2021/2024, *Patterns*) — "a particularly common error is to do feature selection on the whole data set before splitting off the test set"; Apicella §4.4.4 (feature-engineering leakage), citing a review where **15.9%** of ADHD-classification studies applied feature selection to the whole dataset; Kapoor & Narayanan **L1.3**. arXiv

A3. Scaling/normalization on full data (already in MLGuard) — extend to all reduce-style transformers. scikit-learn's general rule: "never call `fit` on the test data"; "the average should be the average of the train subset, **not** the average of all the data." Generalize MLGuard's scaler rule to any estimator exposing `fit`/`fit_transform` upstream of the split or outside the CV Pipeline. scikit-learn

A4. Inconsistent train/test preprocessing. Training scaled/encoded but test not transformed (or re-`fit_transform`-ed on test). scikit-learn: use `fit_transform` on train, `transform` on test. MLGuard already flags `fit_transform` on test; add the converse — a transformer fitted on train but never applied to test before `predict`. *Detectability:* STATIC. scikit-learn

Category B — Improper cross-validation (mostly STATIC)

B1. Preprocessing outside the CV loop. `scaler.fit_transform(X)` → `cross_val_score(model, X_scaled, y)` leaks because every fold's "test" portion influenced the scaler. Fix: pass a `Pipeline` to `cross_val_score`/`cross_validate` (scikit-learn Common Pitfalls). STATIC.

B2. Non-nested CV for hyperparameter tuning, then reporting that score as performance. Using the same CV to both tune and report (`gs = GridSearchCV(...).fit(X,y); print(gs.best_score_)`) is optimistically biased. Cawley & Talbot (JMLR 11:2079–2107, 2010) show the bias can be "surprisingly large." Fix: nested CV (`cross_val_score(GridSearchCV(...), X, y)`). Detectable: `best_score_`/`best_estimator_.score` reported without an outer CV loop. STATIC (heuristic). The scikit-learn "Nested versus non-nested cross-validation" example documents the recommended pattern.

B3. Ignoring grouping structure in CV. Using `KFold`/`StratifiedKFold` when a group column exists → use `GroupKFold`/`StratifiedGroupKFold`/`LeaveOneGroupOut`. STATIC if a group/id variable is detectable; otherwise a heuristic warning.

B4. Random CV for time-series. Using `KFold`/shuffled `cross_val_score` on time-ordered data instead of `TimeSeriesSplit`. STATIC if a datetime index/ordering is detectable; otherwise heuristic. In finance, even `TimeSeriesSplit` is insufficient without **purging + embargo** (López de Prado, *Advances in Financial Machine Learning*, 2018; combinatorial purged CV). Maps to **L3.1** (Kapoor & Narayanan's civil-war case study includes k-fold CV on temporal data as a leakage source). Wikipedia arxiv

B5. Multi-test leakage / repeated test-set reuse. Evaluating repeatedly on the same held-out set and making modeling choices on it converts it into validation data; with no truly untouched test set, performance is optimistic. Dwork et al. (NeurIPS 2015, "Generalization in Adaptive Data Analysis and Holdout Reuse") formalize that adaptive holdout reuse degrades validity (worst-case bias ~√(k/n) for k adaptive queries). Yang et al. detect "multi-test leakage" when only repeatedly-used validation data exists and no independent test set is present. STATIC (data-flow: same test variable feeds ⩾2 evaluations, or evaluation inside a loop). arxiv

Category C — Improper data splitting (STATIC)

C1. Missing `stratify=y` in classification splits (already in MLGuard). Reaffirmed; keep.

C2. Group/entity leakage (random split of multi-record entities). Canonical case: ~3–4 images per patient in ChestX-ray14 split randomly so the model memorizes patients rather than learning pathology. Fix: `GroupShuffleSplit`. Widely documented in medical imaging (Roberts et al., *Nature Machine Intelligence* 3:199–217, 2021; "How You Split Matters," arXiv 2309.00350). STATIC where an id/group column is present. Maps to **L3.2**.

C3. Duplicate / augmentation / oversampling before split (L1.4). Augmenting or oversampling, or leaving duplicate rows, before splitting puts near-identical rows on both sides. STATIC when

augmentation/resampling precedes the split call.

C4. No test set at all (L1.1) / evaluating on training data. `model.fit(X,y)` then `model.score(X,y)`/`predict(X_train)` reported as performance. Yang et al.: **8.0%** of all trained models were evaluated only on data overlapping training; **35.0%** of trained models lacked an independent test set. STATIC (same variable to fit and score). arxiv arxiv

Category D — Preprocessing/encoding correctness (STATIC)

D1. Ordinal encoding of nominal variables. `LabelEncoder`/`OrdinalEncoder` applied to unordered categories (color, city) injects a fake ordinal relationship that linear/distance/tree-threshold models exploit. Also: `LabelEncoder` misused on feature matrices (it is intended for the target `y`). Fix: `OneHotEncoder` for nominal; `OrdinalEncoder(categories=[…])` with an explicit order for true ordinals. STATIC.

D2. Target/mean encoding without cross-fitting (target leakage). Encoding a category by the mean of `y` using the same rows it will train on leaks the target. scikit-learn's `TargetEncoder.fit_transform` performs internal K-fold cross-fitting; using `fit().transform()` on the same data, or a manual `groupby().mean()` merge, disables protection and causes "catastrophic overfitting," especially for high-cardinality features (scikit-learn "Target Encoder's Internal Cross fitting" example). STATIC (detect groupby-mean-on-target merged into features; detect `TargetEncoder.fit` then `.transform` on the *same* training data outside cross-fitting). scikit-learn

Category E — Class-imbalance handling (STATIC)

E1. SMOTE/resampling before split or inside `sklearn.Pipeline`. `SMOTE().fit_resample(X,y)` before `train_test_split`/`cross_val_score` leaks; the test fold then contains synthetic points derived from neighbors. Must use `imblearn.pipeline.Pipeline` so resampling is applied to training folds only (never the test fold). scikit-learn's own `Pipeline` is wrong here because SMOTE exposes `fit_resample`, not `transform`. The imbalanced-learn "Common pitfalls and recommended practices" docs demonstrate the optimistic-bias-then-fix explicitly. STATIC (resampler called outside an imblearn Pipeline, or before the split). Maps to **L1.4**. arxiv

E2. Resampling the test set. Applying SMOTE/under-sampling to evaluation data fabricates the metric. STATIC (resample applied to `X_test`/inside the CV test fold).

E3. Accuracy on imbalanced data. `accuracy_score` on skewed classes (the accuracy paradox). Prefer balanced accuracy, F1/F-beta, MCC, or PR-AUC. *Nuance to encode honestly:* a 2024 peer-reviewed analysis (PMC11240176) shows ROC-AUC is robust to class imbalance while PR-AUC is sensitive to it — so MLGuard should flag bare accuracy on imbalanced targets but **not** dogmatically prescribe one alternative. RUNTIME to confirm imbalance (needs the class distribution); STATIC to flag `accuracy_score`/`scoring='accuracy'` as the *only* metric.

E4. Wrong averaging for multiclass. `average='micro'` on imbalanced multiclass silently masks minority-class performance; the `'macro'` vs `'weighted'` choice can change conclusions. STATIC (flag micro/default averaging on multiclass without justification).

Category F — Reproducibility / randomness (STATIC)

F1. Missing `random_state` (already in MLGuard) on estimators, splitters, and `train_test_split`. Extend to: inconsistent seeds across split/model/CV, and passing a `RandomState` *instance* to a CV splitter — a subtle non-determinism the scikit-learn "Controlling randomness" page warns about ("passing integers to CV splitters is usually the safest option and is preferable"). STATIC. scikit-learn

Category G — Feature/target correctness & illegitimate features

G1. Target column included in features (L2 / target leakage). `X = df.drop(...)` that fails to drop the label, or a feature that is a deterministic proxy of the target (e.g., the `colwars`/`cowwars`/`sdwars` proxies in the civil-war case that produced near-perfect accuracy in Kapoor & Narayanan's reproduction). STATIC for the literal "target in X" case; the proxy case generally needs domain knowledge (RUNTIME/manual). Maps to **L2**. arxiv

G2. Suspiciously perfect / near-perfect scores. AUC/accuracy ≈ 1.0 is a strong leakage signal (Kapoor & Narayanan's case studies; the "too good to be true" heuristic). RUNTIME (needs the computed metric); MLGuard can emit a post-run warning when a reported score exceeds a configurable threshold.

G3. Clever Hans / shortcut learning & confounders. Models exploit spurious correlates (hospital ID, scanner type, rulers in dermoscopy images, background texture) — Geirhos et al., *Nature Machine Intelligence* 2020 ("Shortcut learning in deep neural networks"); Lapuschkin et al., *Nature Communications* 2019; Zech et al. 2018 (pneumonia model keyed on hospital markers). Mostly RUNTIME/data-and-explainability territory, but MLGuard can warn when obvious id/source columns (`patient_id`, `hospital`, `scanner`, `filename`) appear in `X`.

Why these are "silent": the unifying mechanism

Every error above produces a runnable program and plausible numbers. The damage is epistemic: a spurious relationship "that won't be present in the distribution about which scientific claims are made" (Kapoor & Narayanan) inflates the *estimated* performance while leaving syntax and runtime intact. This is exactly why the compiler/linter analogy works — validity, like type-safety, can be partly checked structurally. arxiv

Static vs. runtime summary

- **Statically detectable (code-only):** A1–A4, B1–B5 (B3/B4 heuristic), C1–C4, D1–D2, E1–E2, E4, F1, G1 (literal). These are the priority rule set.

- **Require runtime/data (or heuristic flags):** E3 (need class balance), B3/B4 when no explicit group/time column, G2 (need the score), G3 and proxy-feature G1 (need domain knowledge).

Prevalence evidence (for prioritization)

- **Yang et al. (ASE 2022): 29.6%** of 107,603 GitHub notebooks had ⩾1 leakage; preprocessing 12.3%, overlap 6.5%, multi-test 18.5%; Kaggle notebooks ~**56%** preprocessing leakage; only **5.5%** of notebooks used Pipelines (and **18.1%** of those still leaked). Detection performance: **92.9%** accuracy, **97.6%** precision, **67.8%** recall (262 of 282 potential leakages). Average distance between a preprocessing-leak source and the training call: **293 lines of code** — explaining why cross-cell detection is hard. (arxiv + 3)
- **Kapoor & Narayanan (2023):** leakage in **at least 294 papers across 17 fields** (survey of 22 review papers); correcting it erased the claimed superiority of complex ML over logistic regression in civil-war prediction. (ScienceDirect)
- **Roberts et al. (2021,** *Nature Machine Intelligence* **3:199–217):** of 2,212 studies screened (415 after initial screening) and 62 reviewed in depth, **none of the models were of potential clinical use** due to methodological flaws and/or underlying biases, including leakage and duplicate/"Frankenstein" datasets. (Nature)

Recommendations

Stage 1 — Ship now (high prevalence, clean static signatures, low false-positive risk):

1. Generalize the existing scaler rule into a single **"transformer fitted before split / outside CV pipeline"** rule covering `SimpleImputer`, `KNNImputer`, `IterativeImputer`, `PCA`, `TruncatedSVD`, `SelectKBest`/`SelectPercentile`/`RFE`, `CountVectorizer`/`TfidfVectorizer` (A1–A3, B1).
2. **No-independent-test-set / evaluate-on-training** rule (C4): same variable passed to `.fit` and `.score`/`predict`.
3. **SMOTE/resampler outside an `imblearn` Pipeline or before the split** (E1–E2).
4. **Group leakage**: random split/`KFold` when an id/group column is detectable → recommend `GroupShuffleSplit`/`GroupKFold` (C2).

Stage 2 — Next (need light data-flow or heuristics): 5. **Multi-test / test-set reuse** (B5): same test variable feeds ⩾2 evaluations, or evaluation in a loop. 6. **Non-nested CV bias** (B2): `best_score_` reported without an outer CV loop. 7. **Target/ordinal encoding** rules (D1–D2), including `TargetEncoder.fit().transform()` on the same data and `LabelEncoder` on feature matrices. 8. **Temporal leakage** (B4): random CV when a datetime index/sort is present → `TimeSeriesSplit`.

Stage 3 — Runtime/heuristic add-ons: 9. Post-run **"suspiciously perfect score"** warning (G2) with a configurable threshold. 10. **Accuracy-on-imbalanced** warning (E3) gated on observed

class distribution; suggest balanced accuracy/F1/MCC/PR-AUC without over-prescribing. 11. **Suspicious id/confounder columns** in `X` (G3) — soft warning.

Thresholds that change the staging: If user telemetry shows high false-positive rates on the group-leakage rule (because id columns are hard to infer), demote C2/B3 to Stage 3 heuristics. If Pipeline adoption in your user base is high (>50%), prioritize the "preprocessing leakage *inside* a misused Pipeline" subcase that Yang et al. found in 18.1% of Pipeline users. arxiv

Caveats

- **Some "rules" encode contested guidance.** The metric-for-imbalance question is genuinely debated: the 2024 analysis in PMC11240176 argues ROC-AUC is robust to imbalance and PR-AUC is sensitive, contradicting the common "always use PR-AUC" advice. MLGuard should flag bare accuracy but avoid prescribing a single replacement. nih
- **Static analysis has recall limits.** Yang et al. achieved only **67.8% recall** — leakage spread across many cells/lines (avg. 293 LOC between a preprocessing source and training) is hard to track; MLGuard will miss cross-cell and dynamically constructed cases. arxiv
- **Proxy-feature and Clever Hans leakage (L2, G3) generally require domain knowledge or data/explainability analysis** and cannot be fully resolved by code inspection — keep these as soft, advisory warnings to avoid alarm fatigue.
- **Temporal and group leakage detection depends on inferring data semantics** (is there a time order? an entity id?) that code alone may not reveal; expect heuristic rather than sound detection for B3/B4/C2.
- **Source discipline.** Secondary/blog sources were used only to corroborate code patterns; all load-bearing claims rest on peer-reviewed papers (Kapoor & Narayanan 2023; Kaufman et al. 2012; Lones 2021/2024; Cawley & Talbot 2010; Roberts et al. 2021; Geirhos et al. 2020; Dwork et al. 2015; Varoquaux 2016/2017; Yang et al. 2022; Drobnjaković et al. 2024) and the official scikit-learn and imbalanced-learn documentation. Note one figure correction during research: the widely-cited "329 papers" appears in Kapoor & Narayanan's earlier 2022 preprint, while the peer-reviewed 2023 *Patterns* version reports **294 papers across 17 fields**; this report uses the published figure. arxiv + 2